```
  public void testMultipleSinks()
  {
    MockTimeSink sink2 = new MockTimeSink();
    source.registerObserver(sink2);

    source.setTime(12,13,14);
    assertSinkEquals(sink, 12,13,14);
    assertSinkEquals(sink2, 12,13,14);
  }
}
```

The change needed to make this work is pretty simple. We change `MockTimeSource` to hold all registered observers in a `Vector`. Then, when the time changes, we iterate through the `Vector` and call `update` on all the registered `ClockObservers`. Listings 24-14 and 24-15 show the changes. Figure 24-6 shows the corresponding UML.

Listing 24-14

TimeSource.java

```
public interface TimeSource
{
  public void registerObserver(ClockObserver observer);
}
```

Listing 24-15

MockTimeSource.java

```
import java.util.*;

public class MockTimeSource implements TimeSource
{
  private Vector itsObservers = new Vector();

  public void setTime(int hours, int minutes, int seconds)
  {
    Iterator i = itsObservers.iterator();
    while (i.hasNext())
    {
      ClockObserver observer = (ClockObserver) i.next();
      observer.update(hours, minutes, seconds);

  }

  public void registerObserver(ClockObserver observer)
  {
    itsObservers.add(observer);
  }
}
```

This is pretty nice, but I don't like the fact that the `MockTimeSource` has to deal with the registration and update. It implies that the `Clock` and every other derivative of `TimeSource` will have to duplicate that registration and update code. I don't think `Clock` should have to deal with registration and update. I also don't like the idea of duplicate code. So I'd like to move all that stuff into the `TimeSource`. Of course, this means that `TimeSource`

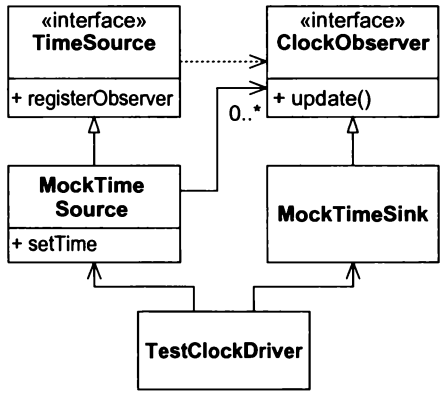


Figure 24-6 Handing multiple `TimeSink` objects

will have to change from an interface to a class. It also means that `MockTimeSource` will shrink to near nothing. Listings 24-16 and 24-17 and Figure 24-7 show the changes.

Listing 24-16

TimeSource.java

```
import java.util.*;

public class TimeSource
{
  private Vector itsObservers = new Vector();

  protected void notify(int hours, int minutes, int seconds)
  {
    Iterator i = itsObservers.iterator();
    while (i.hasNext())
    {
      ClockObserver observer = (ClockObserver) i.next();
      observer.update(hours, minutes, seconds);
    }
  }

  public void registerObserver(ClockObserver observer)
  {
    itsObservers.add(observer);
  }
}
```

Listing 24-17

MockTimeSource.java

```
public class MockTimeSource extends TimeSource
{
  public void setTime(int hours, int minutes, int seconds)
  {
    notify(hours, minutes, seconds);
  }
}
```

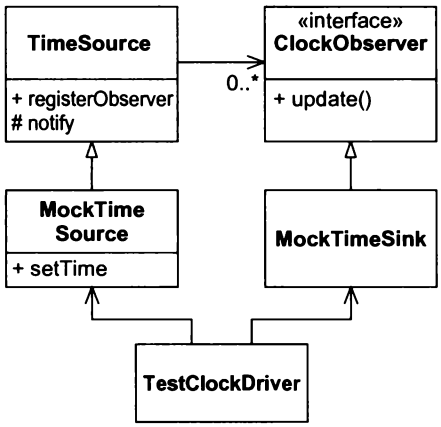


Figure 24-7 Moving registration and update into TimeSource

This is pretty cool. Now, anybody can derive from `TimeSource`. All they have to do to get the observers updated is to call `notify`. But there is still something I don't like about it. `MockTimeSource` inherits directly from `TimeSource`. This means that `Clock` must also derive from `TimeSource`. Why should `Clock` have to depend upon registration and update? `Clock` is just a class that knows about time. Making it depend upon `TimeSource` seems necessary and undesirable.

I know how I'd solve this in C++. I'd create a subclass of both `TimeSource` and `Clock` called `ObservableClock`. I'd override `tic` and `setTime` in `ObservableClock` to call `tic` or `setTime` in `Clock` and then call `notify` in `TimeSource`. See Listing 24-18 and Figure 24-8.

Listing 24-18

`ObservableClock.cc` (C++)

```
class ObservableClock : public Clock, public TimeSource
{
  public:
    virtual void tic()
    {
      Clock::tic();
      TimeSource::notify(getHours(), getMinutes(), getSeconds());
    }

    virtual void setTime(int hours, int minutes, int seconds)
    {
      Clock::setTime(hours, minutes, seconds);
      TimeSource::notify(hours, minutes, seconds);
    }
};
```

Unfortunately, we don't have this option in Java because the language can't deal with multiple inheritance of classes. So, in Java we either have to leave things as they are or use a delegation hack. The delegation hack is shown in Listings 24-19 through 24-21 and Figure 24-9.

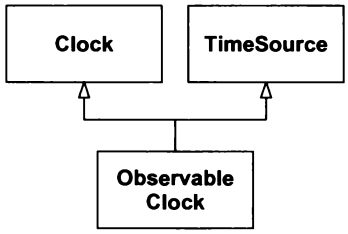


Figure 24-8 Using multiple inheritance in C++ to separate `Clock` from `TimeSource`

Listing 24-19

`TimeSource.java`

```
public interface TimeSource
{
  public void registerObserver(ClockObserver observer);
}
```

Listing 24-20

`TimeSourceImplementation.java`

```
import java.util.*;

public class TimeSourceImplementation
{
  private Vector itsObservers = new Vector();

  public void notify(int hours, int minutes, int seconds)
  {
    Iterator i = itsObservers.iterator();
    while (i.hasNext())
    {
      ClockObserver observer = (ClockObserver) i.next();
      observer.update(hours, minutes, seconds);
    }
  }

  public void registerObserver(ClockObserver observer)
  {
    itsObservers.add(observer);
  }
}
```

Listing 24-21

`MockTimeSource.java`

```
public class MockTimeSource implements TimeSource
{
  TimeSourceImplementation tsImp =
    new TimeSourceImplementation();

  public void registerObserver(ClockObserver observer)
  {
    tsImp.registerObserver(observer);
  }
```

```
  public void setTime(int hours, int minutes, int seconds)
  {
    tsImp.notify(hours, minutes, seconds);
  }
}
```

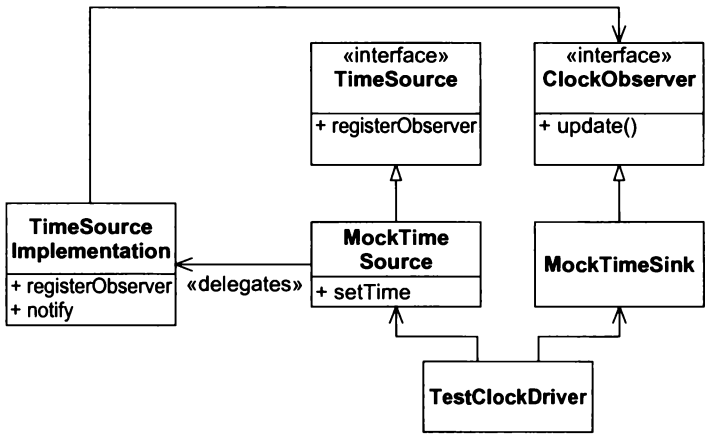


Figure 24-9 `Observer` delegation hack in Java

Notice that the `MockTimeSource` class implements `TimeSource` and contains a reference to an instance of `TimeSourceImplementation`. Notice also that all calls to the `registerObserver` method of `MockTimeSource` are delegated to that `TimeSourceImplementation` object. So too, `MockTimeSource.setTime` invokes `notify` on the `TimeSourceImplementation` instance.

This is ugly, but it has the advantage that `MockTimeSource` does not extend a class. This means that if we were to create `ObservableClock`, it could extend `Clock`, implement `TimeSource`, and delegate to `TimeSourceImplementation`. (See Figure 24-10.) This solves the problem of `Clock` depending upon the registration and update stuff, but at a nontrivial price.

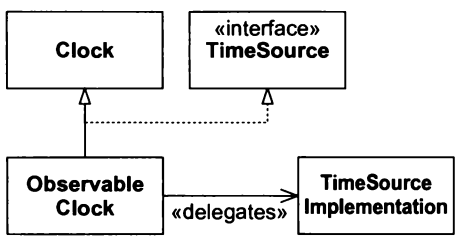


Figure 24-10 The delegation hack for `ObservableClock`

So, let's go back to the way things were in Figure 24-7, before we went down this rathole. We'll simply live with the fact that `Clock` has to depend upon all the registration and update stuff.

`TimeSource` is a stupid name for what the class does. It started out good, back in the days when we had a `ClockDriver`. But things have changed an awful lot since then. We should change the name to something that suggests registration and update. The OBSERVER pattern calls this class `Subject`. Ours seems to be specific to time, so we could call it `TimeSubject`, but that's not a very intuitive name. We could use the old Java moniker `Observable`, but that doesn't ring my chimes either. `TimeObservable`?—No.

Perhaps it is the specificity of the "push-model" observer that is the problem.[2] If we change to a "pull model" we could make the class generic. Then we could change the name of `TimeSource` to `Subject`, and everybody familiar with the OBSERVER pattern would know what it meant.

This is not a bad option. Rather than pass the time in the `notify` and `update` methods, we can have the `TimeSink` ask the `MockTimeSource` for the time. We don't want the `MockTimeSink` to know about the `MockTimeSource`, so we'll create an interface that the `MockTimeSink` can use to get the time. The `MockTimeSource` (and the `Clock`) will implement this interface. We'll call this interface—er—`TimeSource`.

The final state of the code and UML are in Figure 24-11 and Listings 24-22 through 24-27.

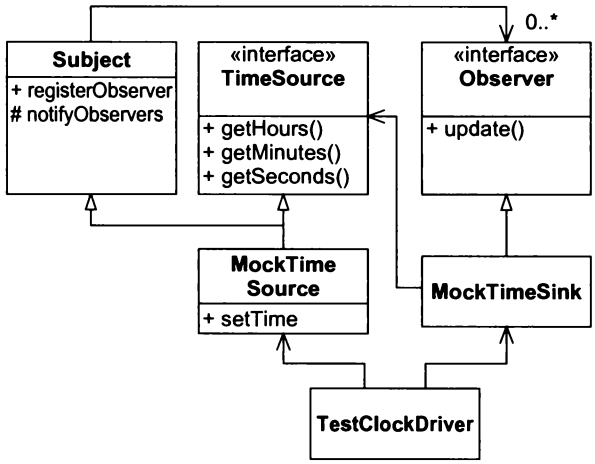


Figure 24-11 Final version of the `Observer` applied to `MockTimeSource` and `MockTimeSink`

Listing 24-22

`ObserverTest.java`

```
import junit.framework.*;

public class ObserverTest extends TestCase
{
  private MockTimeSource source;
  private MockTimeSink sink;

  public ObserverTest(String name)
  {
    super(name);
  }

  public void setUp()
  {
    source = new MockTimeSource();
    sink = new  MockTimeSink(source);
    source.registerObserver(sinkprivate void assertSinkEquals(
  }
```

2. "Push-model" observers push data from the subject to the observer by passing it in the `notify` and `update` methods. "Pull-model" observers pass nothing in the `notify` and `update` methods, and they depend upon the observing object to query the observed object upon receiving an update. See [GOF95].

```
  private void assertSinkEquals(
    MockTimeSink sink, int hours, int minutes, int seconds)
  {
    assertEquals(hours, sink.getHours());
    assertEquals(minutes, sink.getMinutes());
    assertEquals(seconds, sink.getSeconds());
  }

  public void testTimeChange()
  {
    source.setTime(3,4,5);
    assertSinkEquals(sink, 3,4,5);

    source.setTime(7,8,9);
    assertSinkEquals(sink, 7,8,9);
  }

  public void testMultipleSinks()
  {
    MockTimeSink sink2 = new MockTimeSink(source);
    source.registerObserver(sink2);

    source.setTime(12,13,14);
    assertSinkEquals(sink, 12,13,14);
    assertSinkEquals(sink2, 12,13,14);
  }
}
```

Listing 24-23

Observer.java

```
public interface Observer
{
  public void update();
}
```

Listing 24-24

Subject.java

```
import java.util.*;

public class Subject
{
  private Vector itsObservers = new Vector();

  protected void notifyObservers()
  {
    Iterator i = itsObservers.iterator();
    while (i.hasNext())
    {
      Observer observer = (Observer) i.next();
      observer.update();
    }
  }
```

```
  public void registerObserver(Observer observer)
  {
    itsObservers.add(observer);
  }
}
```

Listing 24-25

TimeSource.java

```
public interface TimeSource
{
  public int getHours();
  public int getMinutes();
  public int getSeconds();
}
```

Listing 24-26

MockTimeSource.java

```
public class MockTimeSource extends Subject
                            implements TimeSource
{
  private int itsHours;
  private int itsMinutes;
  private int itsSeconds;

  public void setTime(int hours, int minutes, int seconds)
  {
    itsHours = hours;
    itsMinutes = minutes;
    itsSeconds = seconds;
    notifyObservers();
  }

  public int getHours()
  {
    return itsHours;
  }

  public int getMinutes()
  {
    return itsMinutes;
  }

  public int getSeconds()
  {
    return itsSeconds;
  }
}
```

Listing 24-27

MockTimeSink.java

```
public class MockTimeSink implements Observer
{
  private int itsHours;
```

```
  private int itsMinutes;
  private int itsSeconds;
  private TimeSource itsSource;

  public MockTimeSink(TimeSource source)
  {
    itsSource = source;
  }

  public int getSeconds()
  {
    return itsSeconds;
  }

  public int getMinutes()
  {
    return itsMinutes;
  }

  public int getHours()
  {
    return itsHours;
  }

  public void update()
  {
    itsHours = itsSource.getHours();
    itsMinutes = itsSource.getMinutes();
    itsSeconds = itsSource.getSeconds();
  }
}
```

Conclusion

So, we made it. We started with a design problem and, through reasonable evolution, wound up at the canonical OBSERVER pattern. You might complain that since I knew that I wanted to arrive at the OBSERVER I simply arranged it so that I would. I won't deny it. But that's not really the issue.

If you are familiar with design patterns, then when faced with a design problem, a pattern will very likely pop into your mind. The question then is whether or not to implement that pattern directly, or to continue to evolve the code through a series of small steps. This chapter showed what the second option is like. Rather than simply leaping to the conclusion that the OBSERVER pattern was the best choice for the problem at hand, I continued to resolve the issues one by one. Eventually it was pretty clear that the code was heading in the direction of OBSERVER, so I changed the names and put the code into canonical form.

At any point during that evolution, I could have found that my problem was solved and stopped evolving. Or, I might have found that I could solve the problem by changing course and going in a different direction.

The Use of Diagrams in this Chapter

Some of the diagrams I drew for your benefit. I thought it would be easier for you to follow what I was doing by showing you an overview in a diagram. Had I not been trying to expose and expound, I would not have created them. However, *a few* of the diagrams were created for *my* benefit. There were times when I just needed to stare at the structure that I had created so I could see where to go next.

Had I not been writing a book, I would have drawn these diagrams by hand on a scrap of paper or a whiteboard. I would not have taken the time to use a drawing tool. There are no circumstances that I know of where using a drawing tool is faster than a napkin.

Having used the diagrams to help me evolve the code, I would not have kept the diagrams. In every case, the ones I drew for myself were intermediate steps.

Is there value in keeping diagrams at this level of detail? Clearly, if you are trying to expose your reasoning, as I am doing in this book, they come in pretty handy. But usually we are not trying to document the evolutionary path of a few hours of coding. Usually, these diagrams are transient and are better thrown away. At *this* level of detail, the code is generally good enough to act as its own documentation. At higher levels, that is not always true.

The OBSERVER Pattern

OK, so now that we've been through the example and evolved our code to the OBSERVER pattern, it might be interesting to study just what the OBSERVER pattern is. The canonical form of OBSERVER is shown in Figure 24-12. In this example, `Clock` is being observed by `DigitalClock`. `DigitalClock` registers with the `Subject` interface of `Clock`. `Clock` calls the `notify` method of `Subject` whenever the time changes for any reason. The `notify` method of `Subject` invokes the `update` method of each registered `Observer`. Thus, `DigitalClock` will receive an `update` message whenever the time changes. It uses that opportunity to ask `Clock` for the time and then display it.

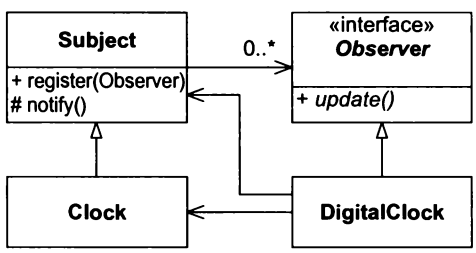


Figure 24-12 Canonical Pull-Model OBSERVER

OBSERVER is one of those patterns that, once you understand it, you see uses for it everywhere. The indirection is very cool. You can register observers with all kinds of objects rather than writing those objects to explicitly call you. While this indirection is a useful way to manage dependencies, it can easily be taken to extremes. Overuse of OBSERVER tends to make systems very difficult to understand and trace.

Push-me-pull-u. There are two primary models of the OBSERVER pattern. Figure 24-13 shows the *pull model* OBSERVER. It gets its name from the fact that the `DigitalClock` must pull the time information from the `Clock` object after receiving the `update` message.

The advantage of the pull model is its simplicity of implementation and the fact that the `Subject` and `Observer` classes can be standard reusable elements in a library. However, imagine that you are observing an employee record with a thousand fields and that you have just received an `update` message. Which of the thousand fields changed?

When `update` is called on the `ClockObserver`, the response is obvious. The `ClockObserver` needs to pull the time from the `Clock` and display it. But when `update` is called on the `EmployeeObserver`, the response is not so obvious. We don't know what happened. We don't know what to do. Perhaps the employee's name changed, or maybe it was his salary. Maybe he got a new boss. Or maybe his bank account changed. We need help.

This help can be given in the push-model form of the OBSERVER pattern. The structure of the push-model observer is shown in Figure 24-13. Notice that the `notify` and `update` methods both take an argument. The argument is a hint, passed from `Employee` to `SalaryObserver` through the `notify` and `update` methods. That hint tells `SalaryObserver` the kind of change the `Employee` record experienced.

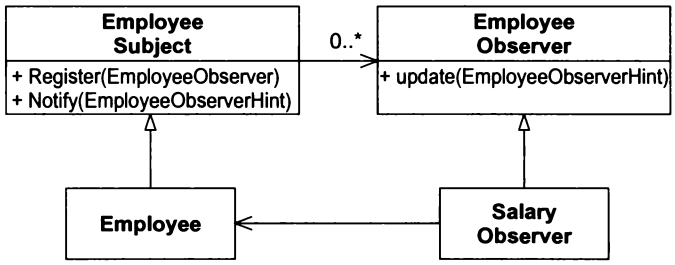


Figure 24-13 Push-Model OBSERVER

The `EmployeeObserverHint` argument of `notify` and `update` might be an enumeration of some kind, a string, or a more complex data structure that contains the old and new values of some field. Whatever it is, its value is being pushed toward the observer.

Choosing between the two different OBSERVER models is simply a matter of the complexity of the observed object. If the observed object is complex, and the observer needs a hint, then the push model is appropriate. If the observed object is simple, then a pull model will do fine.

How OBSERVER Manages the Principles of OOD

The principle that most drives the OBSERVER pattern is the Open–Closed Principle (OCP). The motivation for using the pattern is so that you can add new observing objects without changing the observed object. Thus the observed object stays closed.

Looking back on Figure 24-12, it should be clear that `Clock` is substitutable for `Subject` and that `DigitalClock` is substitutable for `Observer`. Thus, the Liskov Substitution Principle (LSP) is applied.

`Observer` is an abstract class, and the concrete `DigitialClock` depends upon it. The concrete methods of `Subject` also depend upon it. Hence, the Dependency-Inversion Principle (DIP) is applied in this case. You might think that since `Subject` has no abstract methods, the dependency between `Clock` and `Subject` violates the DIP. However, `Subject` is a class that ought never to be instantiated. It only makes sense in the context of a derived class. Thus, `Subject` is *logically* abstract, even though it has no abstract methods. We can enforce the abstractness of `Subject` by giving it a pure virtual destructor in C++ or by making its constructors protected.

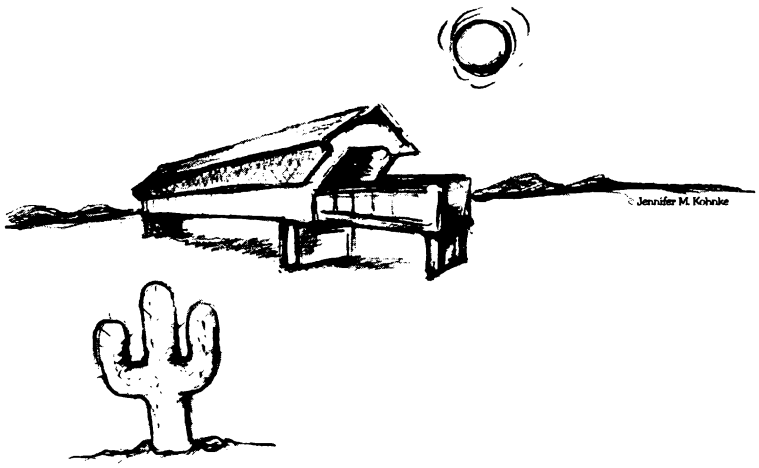
There are hints of the Interface-Segregation Principle (ISP) in Figure 24-11. The `Subject` and `TimeSource` classes segregate the clients of the `MockTimeSource`, providing specialized interfaces for each of those clients.

Bibliography

1. Gamma, et al. *Design Patterns*. Addison–Wesley, 1995.
2. Martin, Robert C., et al. *Pattern Languages of Program Design 3*, Addison–Wesley, 1998.

25

ABSTRACT SERVER, ADAPTER, and BRIDGE



Politicians are the same all over.
They promise to build a bridge even where there is no river.

—Nikita Khrushchev

In the mid-1990s, I was deeply involved with the discussions that coursed through the `comp.object` newsgroup. Those of us who posted messages on that newsgroup argued furiously about different strategies of analysis and design. At one point, we decided that a concrete example would help us evaluate each other's position. So we chose a very simple design problem and proceeded to present our favorite solutions.

The design problem was extraordinarily simple. We chose to design the software that ran inside a simple table lamp. The table lamp has a switch and a light. You could ask the switch whether it was on or off, and you could tell the light to turn on or off. A nice, simple problem.

The debate raged for months. Each person demanded that his own particular style of design was superior to all the others. Some used a simple approach of just a switch and light object. Others thought there ought to be a lamp object that contained the switch and the light. Still others thought that electricity should be an object. One person actually suggested a power-cord object.

Despite the absurdity of most of those arguments, the design model is actually interesting to explore. Consider Figure 25-1. We can certainly make this design work. The `Switch` object can poll the state of the actual switch and can send appropriate `turnOn` an `turnOff` messages to the `Light`.

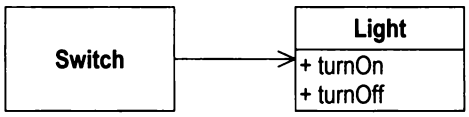


Figure 25-1 Simple table lamp

What don't we like about this design?

Two of our design principles are being violated by this design: the Dependency-Inversion Principle (DIP) and the Open-Closed Principle (OCP). The violation of the DIP is easy to see, the dependency from `Switch` to `Light` is a dependency upon a concrete class. DIP tells us to prefer dependencies on abstract classes. The violation of OCP is a little less direct, but is more to the point. We don't like this design because it forces us to drag a `Light` along everywhere we need a `Switch`. `Switch` cannot be easily extended to control objects other than `Light`.

ABSTRACT SERVER

You might be thinking that you could inherit a subclass from `Switch` that would control something other than a light as in Figure 25-3. But this doesn't solve the problem because `FanSwitch` still inherits the dependency upon `Light`. Wherever you take a `FanSwitch`, you'll have to bring `Light` along. In any case, that particular inheritance relationship also violates the DIP.

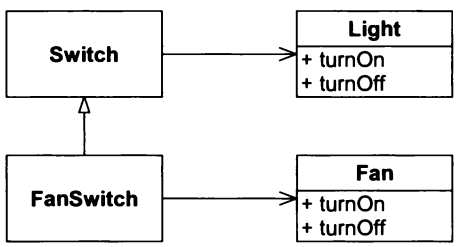


Figure 25-2 A bad way to extended `Switch`

To solve the problem, we invoke one of the simplest of all design patterns: ABSTRACT SERVER. (See Figure 25-2.) By introducing an interface between the `Switch` and the `Light`, we have made it possible for `Switch` to control anything that implements that interface. This immediately satisfies both the DIP and the OCP.

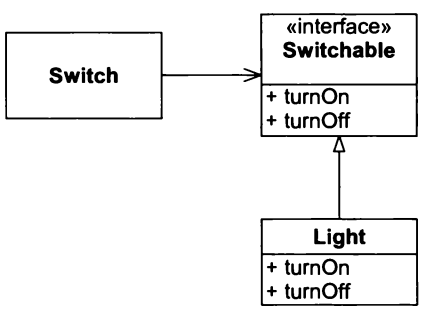


Figure 25-3 ABSTRACT SERVER solution to the Table Lamp problem

Who Owns the Interface?

As an interesting aside, notice that the interface is named for its client. It is called `Switchable` rather than `ILight`. We've talked about this before, and we'll probably notice it again. Interfaces belong to the client, not to the derivative. The logical binding between the client and the interface is stronger than the logical binding between

the interface and its derivatives. It is so strong that it makes no sense to deploy `Switch` without `Switchable`; yet it makes perfect sense to deploy `Switchable` without `Light`. The strength of the logical bonds is at odds with the strength of the physical bonds. Inheritance is a much stronger physical bond than association.

In the early 1990s, we used to think that the physical bond ruled. There were very reputable books that recommended that inheritance hierarchies be placed together in the same physical package. This seemed to make sense because inheritance is such a strong physical bond. But over the last decade, we have learned that the physical strength of inheritance is misleading and that inheritance hierarchies should usually not be packaged together. Rather, clients tend to be packaged with the interfaces they control.

This misalignment of the strength of logical and physical bonds is an artifact of statically typed languages like C++ and Java. Dynamically typed languages, like Smalltalk, Python, and Ruby, don't have the misalignment because they don't use inheritance to achieve polymorphic behavior.

Adapter

There is a problem with the design in Figure 25-3. There is a potential violation of the Single-Responsibility Principle (SRP). We have bound together two things, `Light` and `Switchable`, that may not change for the same reasons. What if we can't add the inheritance relationship to `Light`? What if we purchased `Light` from a third party and we don't have the source code. Or what if there is some other class that we want a `Switch` to control but that we can't derive from `Switchable`? Enter the ADAPTER.[1]

Figure 25-4 shows how the `Adapter` pattern can be employed to solve the problem. The adapter derives from `Switchable` and delegates to `Light`. This solves the problem neatly. Now we can have any object that can be turned on or off controlled by a `Switch`. All we need to do is create the appropriate adapter. Indeed, the object need not even have the same `turnOn` and `turnOff` methods that `Switchable` has. The adapter can be *adapted* to the interface of the object.

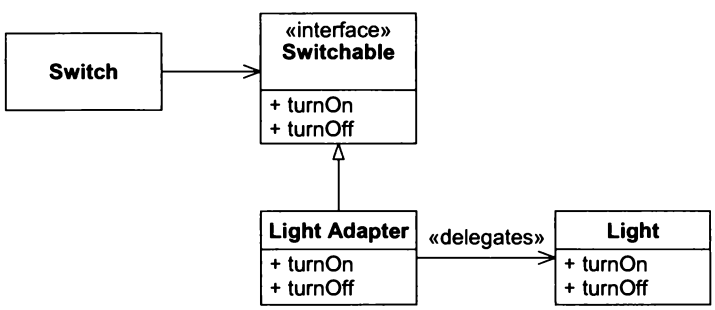


Figure 25-4 Solving the Table Lamp with ADAPTER

TANSTAAFL. Adapters don't come cheap. You need to write the new class, and you need to instantiate the adapter and bind the adapted object to it. Then, every time you invoke the adapter, you have to pay for the time and space required for the delegation. So clearly, you don't want to use adapters all the time. The ABSTRACT SERVER solution is quite appropriate for most situations. In fact, even the initial solution in Figure 25-1 is pretty good unless you happen to *know* that there are other objects for `Switch` to control.

The Class Form of ADAPTER

The `LightAdapter` class in Figure 25-4 is known as an *object form adapter*. There is another approach known as the *class form adapter*, which is shown in Figure 25-5. In this form, the adapter object inherits from both the `Switchable` interface and the `Light` class. This form is a tiny bit more efficient than the object form and is a bit easier to use, but at the expense of using the high coupling of inheritance.

1. We've seen the ADAPTER before, back in Figures 10-2 and 10-3 starting on page 118.

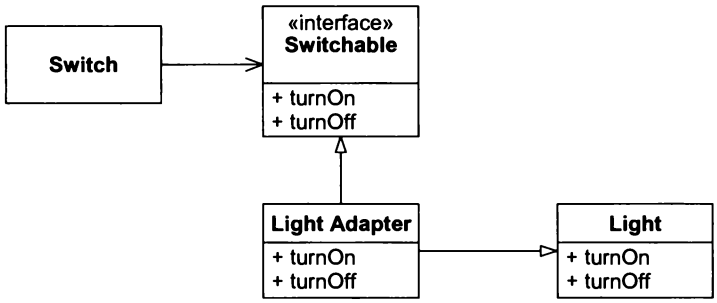


Figure 25-5 Solving the Table Lamp with ADAPTER

The Modem Problem, ADAPTERS and LSP

Consider the situation in Figure 25-6. We have a large number of modem clients all making use of the `Modem` interface. The `Modem` interface is implemented by several derivatives, including `HayesModem`, `USRoboticsModem` and `EarniesModem`. This is a pretty common situation. It conforms nicely to the OCP, LSP, and DIP. Modem clients are unaffected when there are new kinds of modems to deal with. Suppose this situation were to continue for several years. Suppose that there were hundreds of modem clients all making happy use of the `Modem` interface.

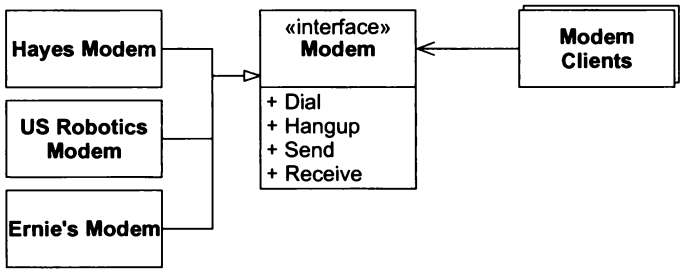


Figure 25-6 Modem Problem

Now suppose that our customers have given us a new requirement. There are certain kinds of modems that don't dial. These are called dedicated modems because they sit at both ends of a dedicated connection.[2] There are several new applications that use these dedicated modems and don't bother to dial. We'll call these the `DedUsers`. However, our customers want all the current modem clients to be able to use these dedicated modems. They tell us that they don't want to have to modify the hundreds of modem client applications, so those modem clients will simply be told to dial dummy phone numbers.

If we had our druthers, we might want to alter the design of our system as shown in Figure 25-7. We'd make use of the ISP to split the dialling and communications functions into two separate interfaces. The old modems would implement both interfaces, and the modem clients would use both interfaces. The `DedUsers` would use nothing but the `Modem` interface, and the `DedicatedModem` would implement just the `Modem` interface. Unfortunately this requires us to make changes to all the modem clients—something that our customers forbade.

So what do we do? We can't separate the interfaces as we'd like, yet we must provide a way for all the modem clients to use `DedicatedModem`. One possible solution is to derive `DedicatedModem` from `Modem` and to implement the `dial` and `hangup` functions to do nothing, as follows:

2. All modems used to be dedicated. It is only in recent geological epochs that modems took on the ability to dial. In the early Jurassic period, you rented a breadbox-sized modem from the phone company and connected it to another modem through dedicated lines that you also rented from the phone company (life was good for the phone company in the Jurassic). If you wanted to dial, you rented another bread-box-sized unit called an auto dialer.

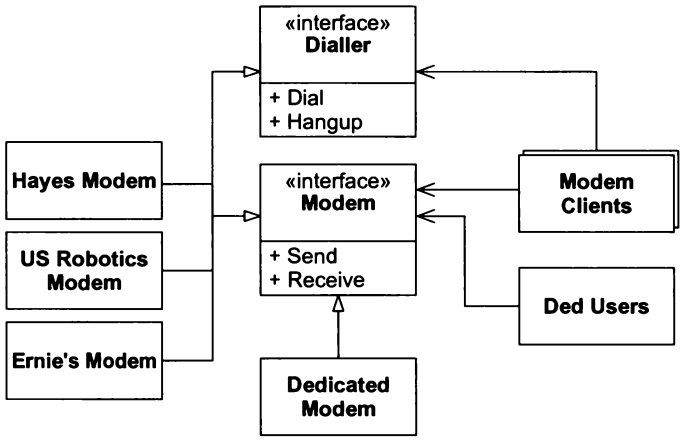


Figure 25-7 Ideal solution to the Modem Problem

```
class DedicatedModem public : Modem
{
  public:
    virtual void dial(char phoneNumber[10]) {}
    virtual void hangup() {}
    virtual void send(char c)
    {...}
    virtual char receive()
    {...}
};
```

Degenerate functions are a sign that we may be violating the LSP. The users of the base class may be expecting `dial` and `hangup` to significantly change the state of the modem. The degenerate implementations in `DedicatedModem` may violate those expectations.

Let's presume that the modem clients were written to expect their modems to be dormant until `dial` is called and to return to dormancy when `hangup` is called. In other words, they don't expect any characters to be coming out of modems that aren't dialled. `DedicatedModem` violates this expectation. It will return characters before `dial` has been called, and it will continue to return them after `hangup` has been called. Thus, `DedicatedModem` may crash some of the modem clients.

Now you might suggest that the problem is with the modem clients. They aren't written very well if they crash on unexpected input. I'd agree with that. But it's going to be hard to convince the folks who have to maintain the modem clients to make changes to their software because we are adding a new kind of modem. Not only does this violate the OCP, it's also just plain frustrating. And besides, our customer has explicitly forbidden us from changing the modem clients.

We Can Fix this with a Kludge. We can simulate a connection status in the `dial` and `hangup` methods of `DedicatedModem`. We can refuse to return characters if `dial` has not been called, or after `hangup` has been called. If we make this change, then all the modem clients will be happy and won't have to change. *All we have to do is convince the* `DedUsers` *to call* `dial` *and* `hangup`. (See Figure 25-8.)

You might imagine that the folks who are building the `DedUsers` find this pretty frustrating. They are explicitly using `DedicatedModem`. *Why should they have to call* `dial` *and* `hangup`*?* However, they haven't written their software yet, so it's easier to get them to do what we want.

A Tangled Web of Dependencies. Months later, when there are hundreds of `DedUsers`, our customers present us with a new change. It seems that all these years our programs have not had to dial international phone

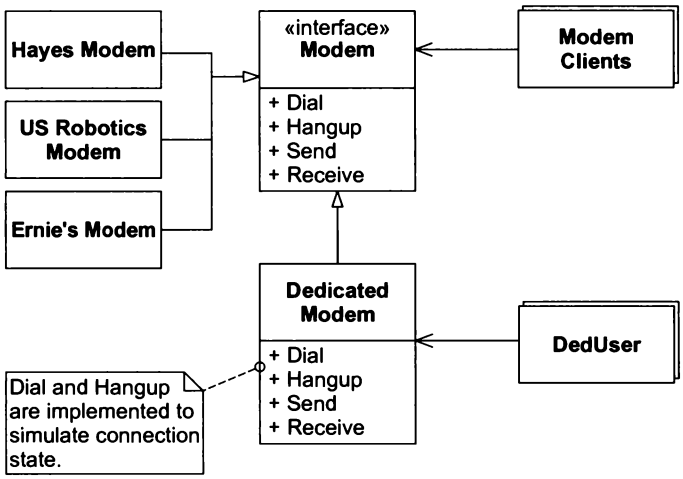


Figure 25-8 Solving the Modem Problem by kluding DedicateModem to simulate connection state

numbers. That's why they got away with the `char[10]` in `dial`. Now, however, our customers want us to be able to dial phone numbers of arbitrary length. They have a need to make international calls, credit-card calls, PIN-identified calls, etc.

Clearly all the modem clients must be changed. They were written to expect `char[10]` for the phone number. Our customers authorize this because they have no choice, and hordes of programmers are put to the task. Just as clearly, the classes in the modem hierarchy must change to accommodate the new phone number size. Our little team can deal with that. *Unfortunately, we now have to go to the authors of the* `DedUsers` *and tell them that they have to change their code!* You might imagine how happy they'll be about that. They aren't calling `dial` because they need to. They are calling `dial` because we told them they have to. And now they are going through an expensive maintenance job because they did what we told them to do.

This is the kind of nasty dependency tangle that many projects find themselves in. A kludge in one part of the system creates a nasty thread of dependency that eventually causes problems in what ought to be a completely unrelated part of the system.

ADAPTER to the Rescue. We could have prevented this fiasco by using an ADAPTER to solve the initial problem as shown in Figure 25-9. In this case, `DedicatedModem` does not inherit from `Modem`. The modem clients use `DedicatedModem` indirectly through the `DedicatedModemAdapter`. This adapter implements `dial` and `hangup` to simulate the connection state. It delegates `send` and `recieve` calls to the `DedicatedModem`.

Note that this eliminates all the difficulties we had before. `Modem` clients are seeing the connection behavior that they expect, and `DedUsers` don't have to fiddle with `dial` or `hangup`. When the phone number requirement changes, the `DedUsers` will be unaffected. Thus, we have fixed both the LSP and OCP violations by putting the adapter in place.

Note that the kludge still exists. The adapter is still simulating connection state. You may think this is ugly, and I'd certainly agree with you. However, notice that all the dependencies point *away* from the adapter. The kludge is isolated from the system, tucked away in an adapter that barely anybody knows about. The only hard dependency upon that adapter will likely be in the implementation of some factory somewhere.[3]

BRIDGE

There is another way to look at this problem. The need for a dedicated modem has added a new degree of freedom to the `Modem` type hierarchy. When the `Modem` type was initially conceived, it was simply an interface for a set of

3. See Chapter 21, FACTORY.

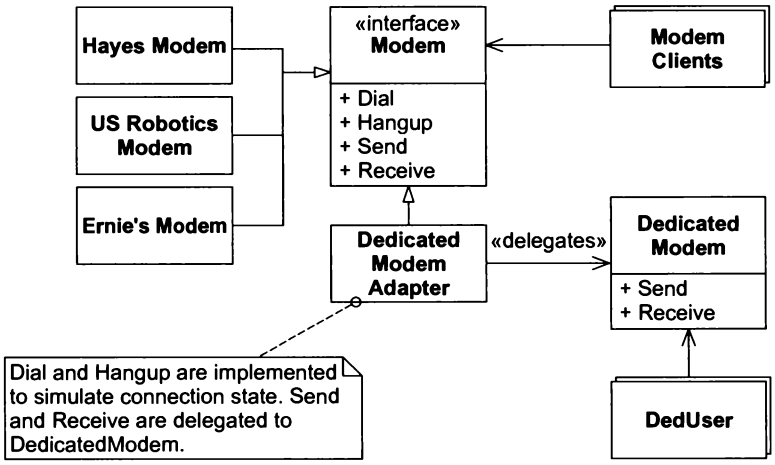


Figure 25-9 Solving the Modem Problem with the ADAPTER

different hardware devices. Thus we had `HayesModem`, `USRModem`, and `ErniesModem` deriving from the base `Modem` class. Now, however, it appears that there is another way to cut at the `Modem` hierarchy. We could have `DialModem` and `DedicatedModem` deriving from `Modem`.

Merging these two independent hierarchies can be done as shown in Figure 25-10. Each of the leaves of the type hierarchy puts either a dialup or dedicated behavior onto the hardware it controls. A `DedicatedHayesModem` object controls a Hayes modem in a dedicated context.

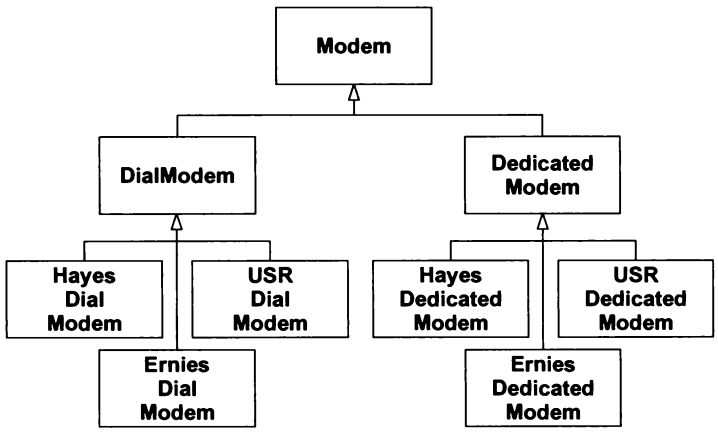


Figure 25-10 Solving the Modem Problem by merging type hierarchies

This is not an ideal structure. Every time we add a new piece of hardware, we must create *two* new classes—one for the dedicated case and one for the dialup case. Every time we add a new connection type, we have to create *three* new classes, one for each of the different pieces of hardware. If these two degrees of freedom are at all volatile, we could wind up with a large number of derived classes before too long.

The BRIDGE pattern often helps in situations where a type hierarchy has more than one degree of freedom. Rather than merge the hierarchies, we can separate them and tie them together with a bridge.

Figure 25-11 shows the structure. We split the modem hierarchy into two hierarchies. One represents the connection method, and the other represents the hardware.

`Modem` users continue to use the `Modem` interface. `ModemConnectionController` implements the `Modem` interface. The derivatives of `ModemConnectionController` control the connection mechanism.

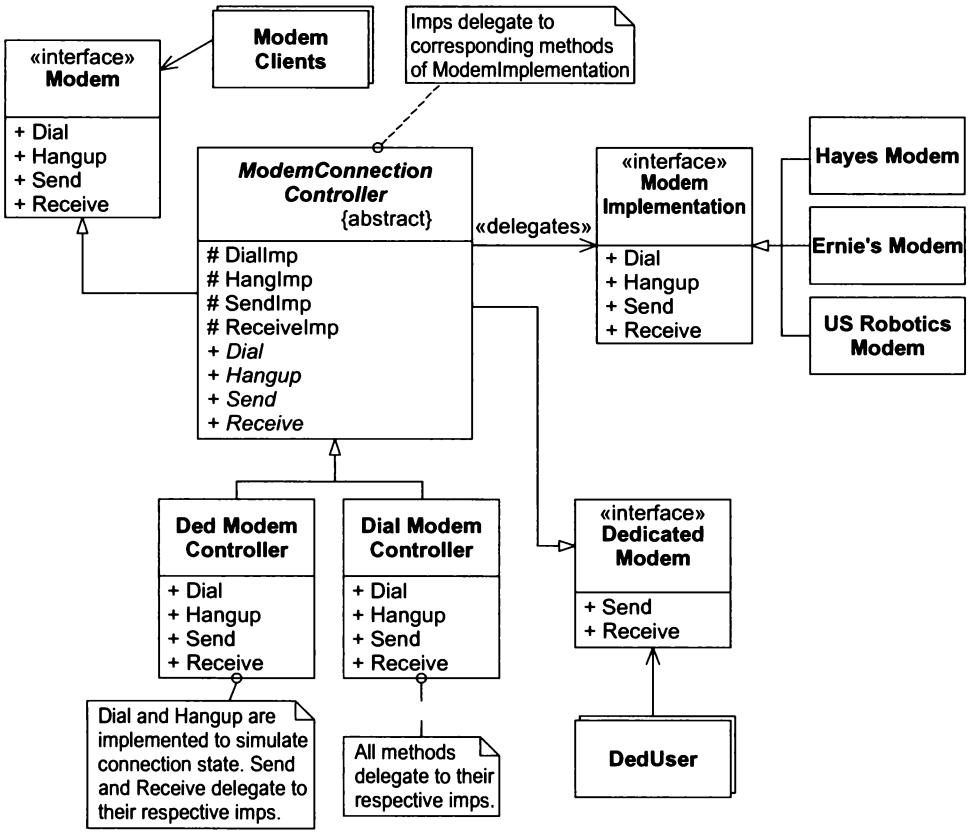


Figure 25-11 BRIDGE solution to the Modem Problem

`DialModemController` simply passes the `dial` and `hangup` method to `dialImp` and `hangImp` in the `ModemConnectionController` base class. Those methods then delegate to the `ModemImplementation` class where they are deployed to the appropriate hardware controller. `DedModemController` implements `dial` and `hangup` to simulate the connection state. It passes `send` and `receive` to `sendImp` and `receiveImp`, which are then delegated to the `ModemImplementation` hierarchy as before.

Note that the four `imp` functions in the `ModemConnectionController` base class are protected. This is because they are strictly to be used by derivatives of `ModemConnectionController`. No one else should be calling them.

This structure is complex, but interesting. We are able to create it without affecting the modem users, and yet it allows us to completely separate the connection policies from the hardware implementation. Each derivative of `ModemConnectionController` represents a new connection policy. That policy can use `sendImp`, `receiveImp`, `dialImp`, and `hangImp` to implement that policy. New `imp` functions could be created without affecting the users. The ISP could be employed to add new interfaces to the connection controller classes. This could create a migration path that the modem clients could slowly follow toward an API that is higher level than `dial` and `hangup`.

Conclusion

One might be tempted to suggest that the real problem with the `Modem` scenario is that the original designers got the design wrong. They should have known that connection and communication were separate concepts. Had they done a little more analysis, they would have found this and corrected it. So it is tempting to blame the problem on insufficient analysis.

Poppycock! There is no such thing as *enough* analysis. No matter how much time you spend trying to figure out the perfect software structure, you will always find that the customer will introduce a change that violates that structure.

There is no escape from this. There are no perfect structures. There are only structures that try to balance the current costs and benefits. Over time those structures must change as the requirements of the system change. The trick to managing that change is to keep the system as simple and flexible as possible.

The ADAPTER solution is simple and direct. It keeps all the dependencies pointing in the right direction, and it's very simple to implement. The BRIDGE solution is quite a bit more complex. I would not suggest embarking down that road until you had very strong evidence that you needed to completely separate the connection and communication policies and that you needed to add new connection policies.

The lesson here, as always, is that a pattern is something that comes with both costs and benefits. You should find yourself using the ones that best fit the problem at hand.

Bibliography

1. Gamma, et al. *Design Patterns*, Reading, MA: Addison–Wesley, 1995.

26

PROXY and STAIRWAY TO HEAVEN: Managing Third Party APIs



Does anybody remember laughter?

—Robert Plant, *The Song Remains the Same*

There are many barriers in software systems. When we move data from our program into the database, we are crossing the database barrier. When we send a message from one computer to another, we are crossing the network barrier.

Crossing these barriers can be complicated. If we aren't careful, our software will be more about the barriers than about the problem to be solved. The patterns in this chapter help us cross such barriers while keeping the program centered on the problem to be solved.

PROXY

Imagine that we are writing a shopping-cart system for a Web site. Such a system might have objects for the customer, the order (the cart), and the products in the order. Figure 26-1 shows a possible structure. This structure is simplistic, but will serve for our purposes.

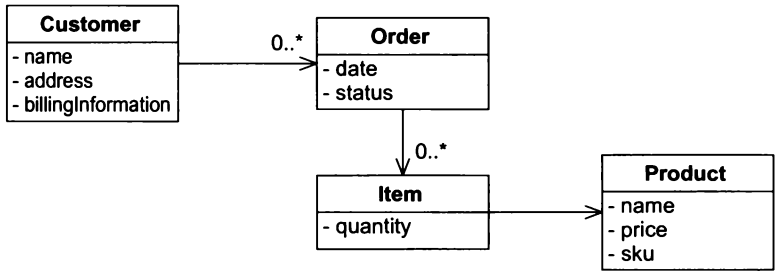


Figure 26-1 Simple shopping cart object model

If we consider the problem of adding a new item to an order, we might come up with the code in Listing 26-1. The `addItem` method of class `Order` simply creates a new `Item` holding the appropriate `Product` and quantity. It then adds that `Item` to its internal `Vector` of `Items`.

Listing 26-1

Adding an item to the Object Model.

```
public class Order
{
  private Vector itsItems = new Vector();
  public void addItem(Product p, int qty)
  {
    Item item = new Item(p, qty);
    itsItems.add(item);
  }
}
```

Now imagine that these objects represent data that are kept in a relational database. Figure 26-2 shows the tables and keys that might represent the objects. To find the orders for a given customer, you find all orders that have the customer's `cusid`. To find all the items in a given order, you find the items that have the order's `orderId`. To find the products referenced by the items, you use the product's `sku`.

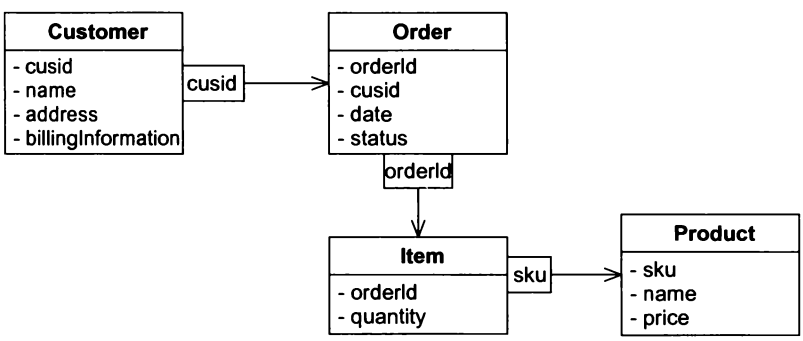


Figure 26-2 Shopping Card Relational Data Model

If we want to add an item row for a particular order, we'd use something like Listing 26-2. This code makes JDBC calls to directly manipulate the relational-data model.

Listing 26-2

Adding an item to the relational model.

```
public class AddItemTransaction extends Transaction
{
  public void addItem(int orderId, String sku, int qty)
  {
    Statement s = itsConnection.CreateStatement();
    s.executeUpdate("insert into items values(" +
                    orderId + "," + sku + "," +
                    qty + ")");
  }
}
```

These two code snippets are very different, and yet they perform the same logical function. They both connect an item to an order. The first ignores the existence of a database, and the second glories in it.

Clearly the shopping cart program is all about orders, items, and products. Unfortunately, if we use the code in Listing 26-2, we make it about SQL statements, database connections, and piecing together query strings. This is a significant violation of the SRP and possibly the CCP. Listing 26-2 mixes together two concepts that change for different reasons. It mixes the concept of the items and orders with the concept of relational schemas and SQL. If either concept must change for any reason, the other concept will be affected. Listing 26-2 also violates the DIP since the policy of the program depends upon the details of the storage mechanism.

The PROXY pattern is a way to cure these ills. To explore this, let's set up a test program that demonstrates the behavior of creating an order and calculating the total price. The salient part of this program is shown in Listing 26-3.

Listing 26-3

Test program creates order and verifies calculation of price.

```
public void testOrderPrice()
  {
    Order o = new Order("Bob");
    Product toothpaste = new Product("Toothpaste", 129);
    o.addItem(toothpaste, 1);
    assertEquals(129, o.total());
    Product mouthwash = new Product("Mouthwash", 342);
    o.addItem(mouthwash, 2);
    assertEquals(813, o.total());
  }
```

The simple code that passes this test is shown in Listings 26-4 through 26-6. It makes use of the simple object model in Figure 26-1. It does not assume that there is a database anywhere. It is also incomplete in many ways. It is just enough code to get the test to pass.

Listing 26-4

order.java

```
public class Order
{
  private Vector itsItems = new Vector();
```

```
  public Order(String cusid)
  {
  }

  public void addItem(Product p, int qty)
  {
    Item item = new Item(p,qty);
    itsItems.add(item);
  }

  public int total()
  {
    int total = 0;
    for (int i = 0; i < itsItems.size(); i++)
    {
      Item item = (Item) itsItems.elementAt(i);
      Product p = item.getProduct();
      int qty = item.getQuantity();
      total += p.getPrice() * qty;
    }
    return total;
  }
}
```

Listing 26-5

product.java

```
public class Product
{
  private int itsPrice;

  public Product(String name, int price)
  {
    itsPrice = price;
  }

  public int getPrice()
  {
    return itsPrice;
  }
}
```

Listing 26-6

item.java

```
public class Item
{
  private Product itsProduct;
  private int itsQuantity;

  public Item(Product p, int qty)
  {
    itsProduct = p;
    itsQuantity = qty;
  }
```

```
  public Product getProduct()
  {
    return itsProduct;
  }

  public int getQuantity()
  {
    return itsQuantity;
  }
}
```

Figures 26-3 and 26-4 show how the PROXY pattern works. Each object that is to be proxied is split into three parts. The first is an interface that declares all the methods that clients need to invoke. The second is a class that implements those methods without knowledge of the database. The third is the proxy that knows about the database.

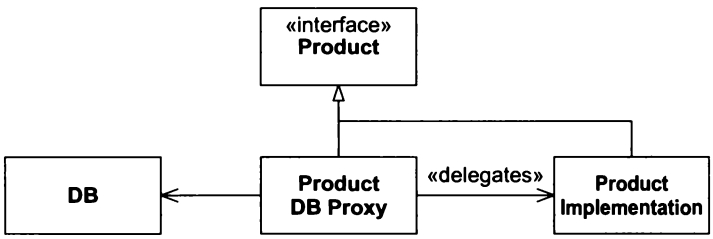


Figure 26-3 PROXY static model

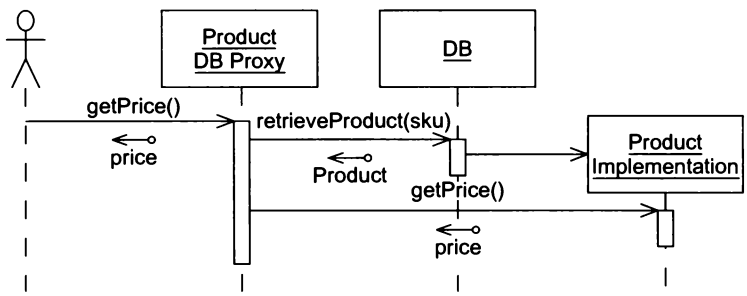


Figure 26-4 PROXY dynamic model

Consider the `Product` class. We have proxied it by replacing it with an interface. This interface has all the same methods that `Product` has. The `ProductImplementation` class implements the interface almost exactly as before. The `ProductDBProxy` implements all the methods of `Product` to fetch the product from the database, create an instance of `ProductImplementation`, and then delegate the message to it.

The sequence diagram in Figure 26-4 shows how this works. The client sends the `getPrice` message to what it thinks is a `Product`, but what is really a `ProductDBProxy`. The `ProductDBProxy` fetches the `ProductImplementation` from the database. It then delegates the `getPrice` method to it.

Neither the client nor the `ProductImplementation` knows that this has happened. The database has been inserted into the application without either party knowing about it. That's the beauty of the PROXY pattern. In theory, it can be inserted in between two collaborating objects without those objects having to know about it. Thus, it can be used to cross a barrier like a database or a network without either of the participants knowing about it.

In reality, using proxies is nontrivial. To get an idea what some of the problems are, let's try to add the PROXY pattern to the simple shopping cart application.

Proxifying the Shopping Cart

The simplest Proxy to create is for the `Product` class. For our purposes, the product table represents a simple dictionary. It will be loaded in one place with all the products. There is no other manipulation of this table, and that makes the proxies relatively trivial.

To get started, we need a simple database utility that stores and retrieves product data. The proxy will use this interface to manipulate the database. Listing 26-7 shows the test program for what I have in mind. Listings 26-8 and 26-9 make that test pass.

Listing 26-7

DBTest.java

```
import junit.framework.*;
import junit.swingui.TestRunner;

public class DBTest extends TestCase
{
  public static void main(String[] args)
  {
    TestRunner.main(new String[]{"DBTest"});
  }

  public DBTest(String name)
  {
    super(name);
  }

  public void setUp() throws Exception
  {
    DB.init();
  }

  public void tearDown() throws Exception
  {
    DB.close();
  }

  public void testStoreProduct() throws Exception
  {
    ProductData storedProduct = new ProductData();
    storedProduct.name = "MyProduct";
    storedProduct.price = 1234;
    storedProduct.sku = "999";
    DB.store(storedProduct);
    ProductData retrievedProduct = DB.getProductData("999");
    DB.deleteProductData("999");
    assertEquals(storedProduct, retrievedProduct);
  }
}
```

Listing 26-8

ProductData.java

```
public class ProductData
{
  public String name;
```

```
  public int price;
  public String sku;

  public ProductData()
  {
  }

  public ProductData(String name, int price, String sku)
  {
    this.name = name;
    this.price = price;
    this.sku = sku;
  }

  public boolean equals(Object o)
  {
    ProductData pd = (ProductData)o;
    return name.equals(pd.name) &&
           sku.equals(pd.sku) &&
           price==pd.price;
  }
}
```

Listing 26-9

DB.java

```
import java.sql.*;

public class DB
{
  private static Connection con;

  public static void init() throws Exception
  {
    Class.forName("sun.jdbc.odbc.JdbcOdbcDriver");
    con = DriverManager.getConnection(
       "jdbc:odbc:PPP Shopping Cart");
  }

  public static void store(ProductData pd) throws Exception
  {
    PreparedStatement s = buildInsertionStatement(pd);
    executeStatement(s);
  }

  private static PreparedStatement
  buildInsertionStatement(ProductData pd) throws SQLException
  {
    PreparedStatement s = con.prepareStatement(
      "INSERT into Products VALUES (?, ?, ?)");
    s.setString(1, pd.sku);
    s.setString(2, pd.name);
    s.setInt(3, pd.price);
    return s;
  }
```

```
public static ProductData getProductData(String sku) throws Exception
{
  PreparedStatement s = buildProductQueryStatement(sku);
  ResultSet rs = executeQueryStatement(s);
  ProductData pd = extractProductDataFromResultSet(rs);
  rs.close();
  s.close();
  return pd;
}

private static PreparedStatement
buildProductQueryStatement(String sku) throws SQLException
{
  PreparedStatement s = con.prepareStatement(
    "SELECT * FROM Products WHERE sku = ?;");
  s.setString(1, sku);
  return s;
}

private static ProductData
extractProductDataFromResultSet(ResultSet rs) throws SQLException
{
  ProductData pd = new ProductData();
  pd.sku = rs.getString(1);
  pd.name = rs.getString(2);
  pd.price = rs.getInt(3);
  return pd;
}

public static void deleteProductData(String sku) throws Exception
{
  executeStatement(buildProductDeleteStatement(sku));
}

private static PreparedStatement
buildProductDeleteStatement(String sku) throws SQLException
{
  PreparedStatement s = con.prepareStatement(
    "DELETE from Products where sku = ?");
  s.setString(1, sku);
  return s;
}

private static void executeStatement(PreparedStatement s) throws SQLException
{
  s.execute();
  s.close();
}

private static ResultSet
executeQueryStatement(PreparedStatement s)
```

```
  throws SQLException
  {
    ResultSet rs = s.executeQuery();
    rs.next();
    return rs;
  }

  public static void close() throws Exception
  {
    con.close();
  }
}
```

The next step in implementing the proxy is to write a test that shows how it works. This test adds a product to the database. It then creates a `ProductProxy` with the `sku` of the stored product and attempts to use the accessors of `Product` to acquire the data from the proxy. (See Listing 26-10.)

Listing 26-10

ProxyTest.java

```
import junit.framework.*;
import junit.swingui.TestRunner;

public class ProxyTest extends TestCase
{
  public static void main(String[] args)
  {
    TestRunner.main(new String[]{"ProxyTest"});
  }

  public ProxyTest(String name)
  {
    super(name);
  }

  public void setUp() throws Exception
  {
    DB.init();
    ProductData pd = new ProductData();
    pd.sku = "ProxyTest1";
    pd.name = "ProxyTestName1";
    pd.price = 456;
    DB.store(pd);
  }

  public void tearDown() throws Exception
  {
    DB.deleteProductData("ProxyTest1");
    DB.close();
  }

  public void testProductProxy() throws Exception
  {
    Product p = new ProductProxy("ProxyTest1");
    assertEquals(456, p.getPrice());
```